



PhysEZ

Multilingual Physics Tutoring, Booking and Student Management Platform

Prepared as a pre-development design plan — what will be built, why
it is needed, and how the system will work.

STUDENT NAME

Karam Shekh Yusuf

STUDENT ID

208252817

VERSION

1.0

DATE

August 2026

00 Document Map

A concise plan of what will be built, why it is needed, and how the system will work.

#	Section	What it covers
1	Overview, Goals & Scope	Product summary, problem, goals, non-goals, success metrics
2	Requirements	Functional and testable non-functional requirements
3	System Architecture & Stack	Components, responsibilities, technology choices, integration flow
4	Data Model	Entities, relationships, schema decisions, indexes and constraints
5	API Design	REST endpoints, request/response examples, error contract
6	Frontend Design	Student/admin routes, components, state and localization
7	Security & Non-Functionals	Authentication, authorization, validation, file and webhook security
8	Testing & Deployment	Test levels, Netlify/VPS deployment, configuration and backups
9	Timeline, Risks & Appendix	Milestones, risks, future features, open questions and sources

01 Overview, Goals & Scope

Setting the product boundary before implementation.

Summary

PhysEZ is a multilingual web platform for private physics tutoring. Students can create accounts, select a lesson type (Zoom or in-person), duration, language, physics topic, date and available time, submit a booking request, upload supporting material, complete an optional diagnostic quiz, and later view lesson summaries and progress. A teacher/admin reviews requests from a separate admin interface. Confirmed lessons are synchronized to Google Calendar, time availability is updated, and n8n handles automation workflows such as Telegram notifications and scheduled reminders.

Problem statement

Private tutoring often requires several disconnected tools: messaging for requests, a calendar for scheduling, files for questions, and manual tracking for lesson history. This makes double-booking, lost preparation material, inconsistent follow-up, and repeated administrative work more likely. PhysEZ centralizes the workflow in one system while keeping external services such as Google Calendar and Telegram synchronized.

Goals

- Allow students to register, log in, and manage their own tutoring activity.

- Allow students to request Zoom or in-person lessons by duration, language, topic, date and available time.
- Provide a teacher/admin dashboard for reviewing, confirming, rejecting, cancelling and rescheduling requests.
- Prevent double-booking at the backend and keep public availability consistent with confirmed lessons.
- Use a self-hosted n8n server for automation and external integrations, especially Telegram and Google Calendar.
- Let students upload preparation material and optionally complete a topic-based diagnostic quiz.
- Allow the teacher to publish post-lesson summaries, homework and feedback, then show student progress over time.
- Support Arabic, Hebrew and English, including RTL layout for Arabic and Hebrew.
- Deploy the student and admin frontends on Netlify while keeping the application API, database, n8n and files on a VPS.

Non-goals for v1

- Online payment processing and invoices.
- Multi-teacher marketplace or public tutor discovery.
- Native iOS/Android applications.
- Built-in video calling or video hosting; Zoom is treated as a lesson type/link.
- AI-generated tutoring or automatic grading beyond deterministic diagnostic quiz scoring.
- Recurring lesson packages, waiting lists and advanced financial analytics; these are future features.

Why the non-goals matter

Tutoring platforms attract "can it also just..." requests. Fixing the boundary now means the core booking, calendar and authorization work is finished before optional learning features begin.

Success metrics

ID	Success condition
SM1	A new student can register, choose an available slot and submit a valid booking without admin assistance.
SM2	An admin can confirm a pending booking; it becomes unavailable to other students and a calendar event is created.
SM3	Cancelling a confirmed booking removes or updates the external calendar event and reopens the time when appropriate.
SM4	Two overlapping confirmed bookings cannot be created for the teacher, even if requests arrive close together.
SM5	A student can upload a supported file, complete a diagnostic quiz, and later view a lesson summary from the dashboard.
SM6	The student-facing flow works in Arabic, Hebrew and English with correct RTL/LTR direction.

02 Requirements

What the system must do, expressed as a testable checklist.

Functional requirements

ID	Functional requirement
FR1	A visitor can register a student account with name, email and password.
FR2	A registered user can log in and log out; the API exposes the current authenticated user.
FR3	The system supports student and admin roles and enforces role permissions on the backend.
FR4	A student can view available lesson times for a selected date.
FR5	A student can create a booking request with Zoom/in-person type, duration, language, topic, date/time and notes.
FR6	A new booking is stored with status PENDING and triggers an n8n workflow that sends a Telegram notification to the admin.
FR7	The admin can list and filter bookings by status, date and student.
FR8	The admin can confirm or reject a pending booking.
FR9	Before confirmation, the backend re-checks time overlap; a conflicting confirmed booking blocks confirmation.
FR10	Confirmation triggers n8n to create a Google Calendar event and stores the returned event identifier/sync status.
FR11	A confirmed booking is removed from student-visible availability.
FR12	A student can cancel an allowed booking; cancellation updates the database and triggers calendar cleanup.
FR13	A student can request rescheduling; the admin can accept or reject the new time.
FR14	A student dashboard shows pending, confirmed, completed and cancelled lesson history.
FR15	A student can select a physics level/topic and upload PDF/JPG/PNG preparation files up to the configured size limit.
FR16	The teacher can view files belonging to a booking; students can only access their own files.
FR17	A student can take a diagnostic quiz associated with a selected topic and receive an automatically calculated score.
FR18	The teacher can create a post-lesson summary with topics covered, homework, feedback and attendance.
FR19	The student can view lesson summaries and a simple progress history based on completed lessons and quiz scores.
FR20	The interfaces support English, Arabic and Hebrew and switch between LTR and RTL appropriately.
FR21	The admin can create and remove availability slots.
FR22	n8n can send scheduled Telegram reminders to the teacher for upcoming confirmed lessons without exposing its editor publicly.

Non-functional requirements

Category	ID	Requirement
Performance	NFR1	For normal loads, 95% of non-file API requests should respond in under 500 ms, excluding third-party API latency.
Security	NFR2	Passwords are stored only as strong hashes (bcrypt or argon2id); all protected API routes require valid authentication and server-side authorization.
Transport	NFR3	Production traffic uses HTTPS; authentication tokens and secrets are never placed in URLs or committed to Git.
Availability	NFR4	The production API targets 99% monthly availability excluding planned maintenance; failed external integrations do not corrupt booking state.
Accessibility	NFR5	Forms have labels, keyboard navigation works for core flows, and color is not the only indicator of status.
Localization	NFR6	Arabic and Hebrew screens render RTL; English renders LTR; dates/times are shown in the configured locale/time zone.
File safety	NFR7	Uploads are limited to approved MIME types and a configurable maximum size (default 10 MB) and are served only after authorization.
Data integrity	NFR8	The database prevents overlapping confirmed lesson times and maintains valid foreign-key relationships.
Recovery	NFR9	PostgreSQL and uploaded files have scheduled backups, and a restore procedure is documented before final deployment.
Scalability	NFR10	The design supports at least 1,000 student accounts and 50,000 booking/history rows without schema redesign; lists are paginated when needed.

03 System Architecture & Stack

Major components, data flow, responsibilities and the reasons behind each choice.

High-level architecture. Two React applications will be deployed independently on Netlify. Both will call the same Express REST API on the VPS. Express will own authentication, business rules and database state. n8n will serve as the automation/integration layer rather than the primary application backend.

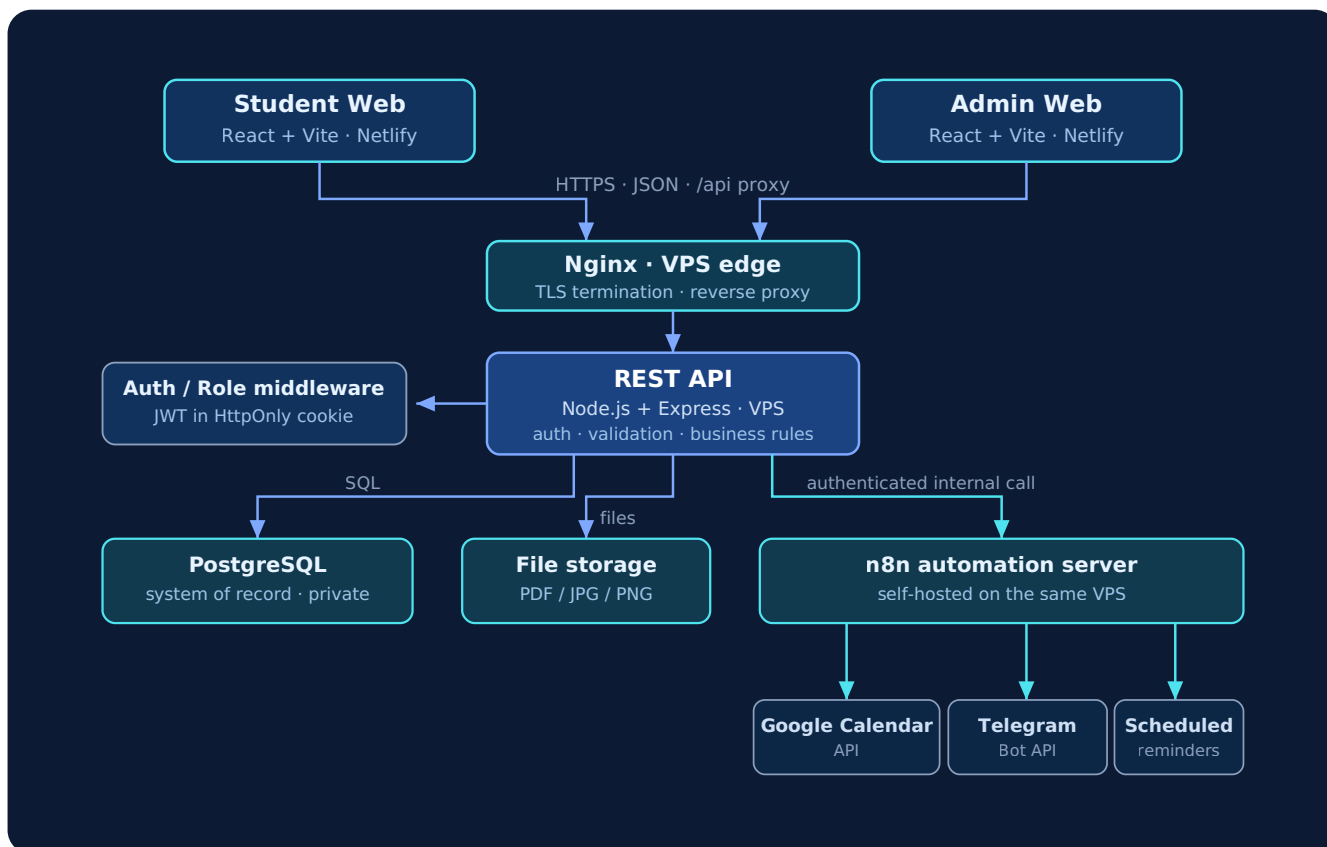


Figure 1. Planned PhysEZ system architecture.

Component responsibilities

Component	Responsibility
Student React SPA	Booking flow, account/dashboard, file upload, diagnostic quiz, lesson history, multilingual UI.
Admin React SPA	Booking review, availability management, student context, lesson summaries, quiz results and operational status.
Express REST API	Business logic, validation, authentication/authorization, booking state transitions, database access and secure file endpoints.
PostgreSQL	System of record for users, bookings, availability, lesson records, quiz data and file metadata.
n8n	External integrations and asynchronous/scheduled automation: Telegram, Google Calendar, reminder workflows and integration retries.
VPS file storage	Stores uploaded PDFs/images in v1; PostgreSQL stores metadata and paths, not file binaries.
Nginx	Terminates/forwards HTTPS traffic to Express and n8n on private internal ports.
Netlify	Will host and deploy the two Vite frontends and provide their public URLs.

Technology stack and rationale

Layer	Choice	Why
Student/Admin frontend	React + Vite	Component-based UI; Vite provides a simple development/build toolchain for an SPA.
Routing	React Router	Clear separation of public, student and admin routes.
Backend	Node.js + Express	REST API with JavaScript across client/server; middleware fits auth, validation and role checks.
Database	PostgreSQL	Relational model with strong keys, constraints, ranges and transaction support for booking integrity.
Authentication	JWT in Secure HttpOnly cookie	Keeps the token out of JavaScript-accessible storage; backend middleware enforces access.
Automation	Self-hosted n8n	Separates integration workflows from core booking logic and supports scheduled automations.
Calendar	Google Calendar API through n8n	Confirmed lessons become calendar events; update/delete operations keep scheduling synchronized.
Notifications	Telegram Bot API through n8n	Immediate admin notification without embedding Telegram logic inside the application core.
Files	VPS filesystem in v1	Simple and sufficient for a course project; later replaceable with S3-compatible object storage.
Frontend hosting	Netlify	Public Netlify URLs, Vite deployment and Git-based frontend delivery.
VPS edge	Nginx + HTTPS certificate tooling	Reverse proxy exposes only HTTPS endpoints while Express, PostgreSQL and n8n remain on private/internal ports.

Why n8n is not the main backend

The application needs to demonstrate full-stack backend work: REST routes, middleware, authentication, role-based authorization, database constraints, validation and business rules. For that reason, Express is the authoritative application backend. n8n handles integration-oriented work that benefits from visual workflows, scheduling and retryability. This keeps the design academically stronger and technically cleaner.

Trade-off

Directly calling Google Calendar and Telegram from Express would reduce one component, but would mix third-party integration code with core booking logic. Using n8n adds operational complexity, but it also isolates external workflows and makes scheduled reminders easier to maintain.

Booking confirmation flow

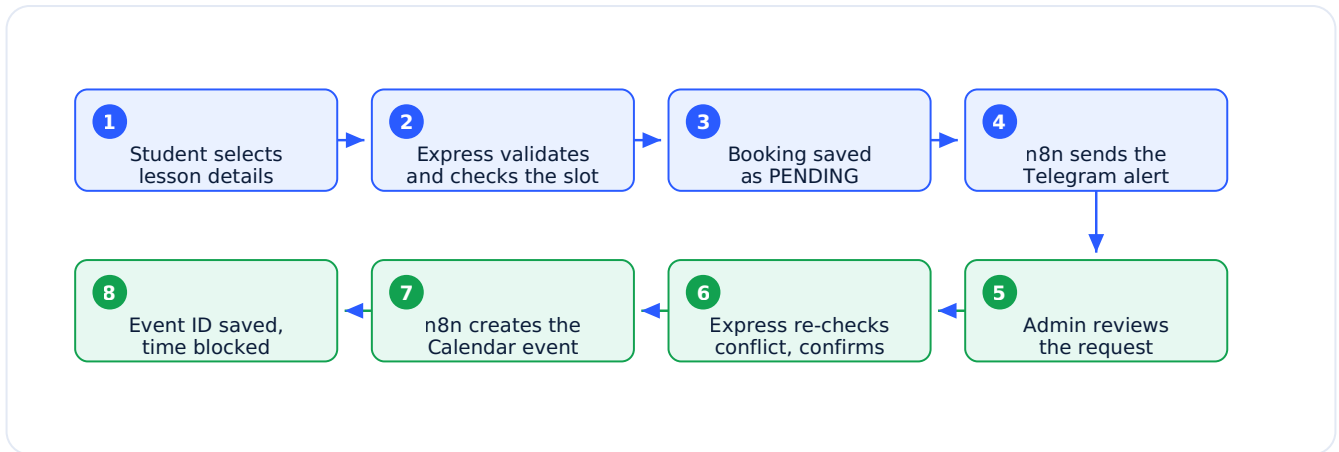


Figure 2. Booking request and confirmation workflow.

Failure handling

If calendar creation fails after a booking is confirmed, the booking remains blocked to prevent a double-booking, `calendar_sync_status` is set to `failed`, the admin sees a retry state, and n8n can retry the calendar workflow.

04 Data Model

Relational entities are defined before routes so the API and UI share a stable source of truth.

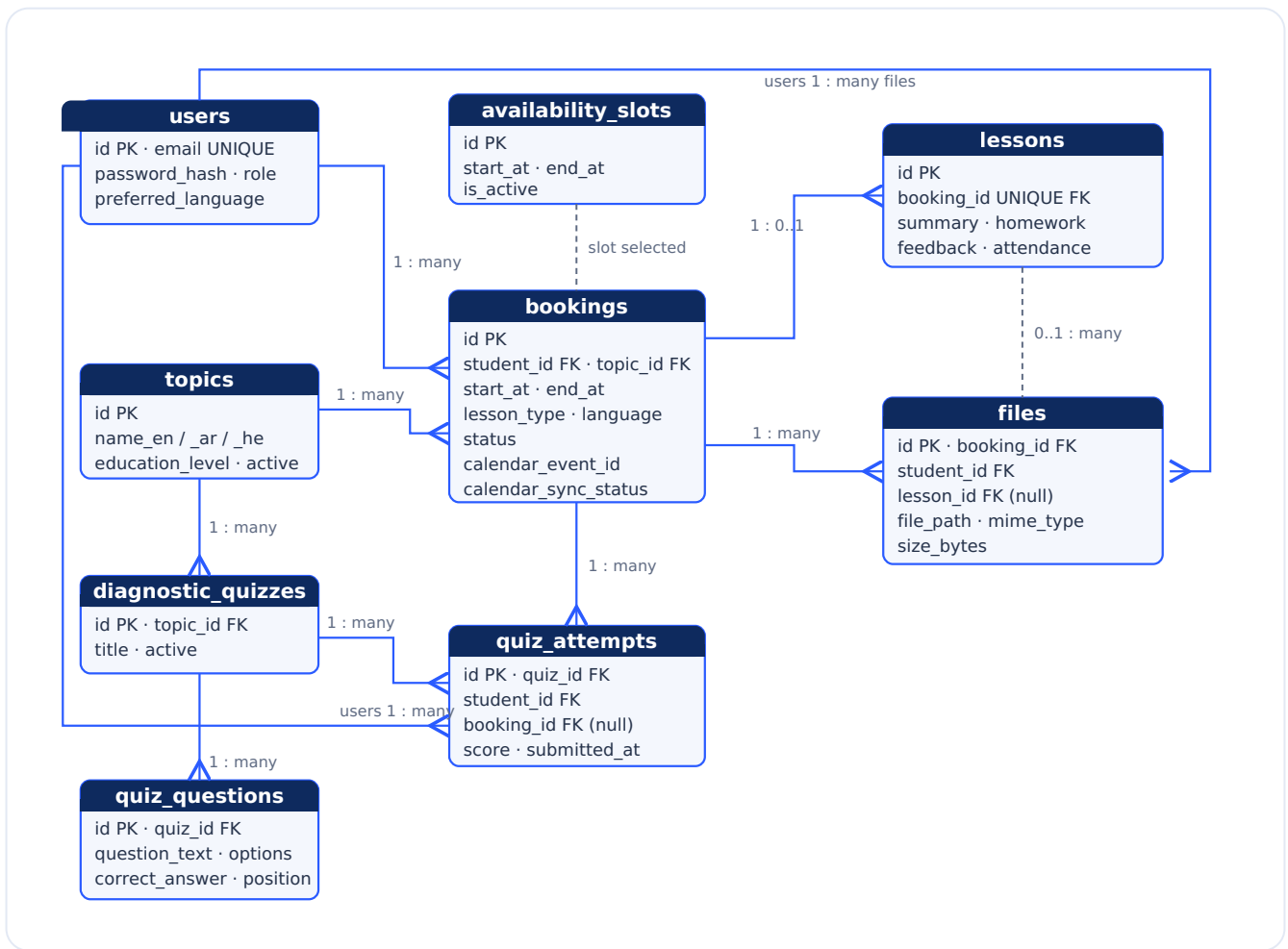


Figure 3. Core entity relationship model.

Core entities

Entity	Fields, data types, keys and constraints
users	id UUID PK; name VARCHAR(120) NOT NULL; email VARCHAR(255) UNIQUE NOT NULL; password_hash VARCHAR(255) NOT NULL; role VARCHAR(20) CHECK (student/admin); preferred_language VARCHAR(5); created_at TIMESTAMPTZ. Referenced by bookings, files and quiz_attempts.
topics	id UUID PK; name_en/name_ar/name_he VARCHAR(120); education_level VARCHAR(80); active BOOLEAN. Referenced by bookings and diagnostic_quizzes.
availability_slots	id UUID PK; start_at TIMESTAMPTZ; end_at TIMESTAMPTZ; is_active BOOLEAN; created_at TIMESTAMPTZ. CHECK end_at > start_at; index start_at/end_at.
bookings	id UUID PK; student_id UUID FK → users; topic_id UUID FK → topics; lesson_type VARCHAR(20); language VARCHAR(5); start_at/end_at TIMESTAMPTZ; status VARCHAR(30) CHECK; notes TEXT; calendar_event_id VARCHAR(255); calendar_sync_status VARCHAR(30); created_at TIMESTAMPTZ. Indexes: student_id, status, start_at, topic_id. Exclusion constraint prevents overlapping confirmed lessons.
lessons	id UUID PK; booking_id UUID UNIQUE FK → bookings; attendance VARCHAR(20); summary TEXT; homework TEXT; feedback TEXT; created_at TIMESTAMPTZ. One lesson record per booking.

Entity	Fields, data types, keys and constraints
files	id UUID PK; booking_id UUID FK → bookings; student_id UUID FK → users; lesson_id UUID NULL FK → lessons (set when the teacher attaches material to a lesson summary); original_name/stored_name VARCHAR(255); file_path TEXT; mime_type VARCHAR(100); size_bytes BIGINT; created_at TIMESTAMPTZ. Indexes: booking_id, student_id. PostgreSQL stores metadata only; bytes remain on VPS storage.
diagnostic_quizzes	id UUID PK; topic_id UUID FK → topics; title VARCHAR(200); active BOOLEAN; created_at TIMESTAMPTZ. A topic may have multiple quiz versions; only active quizzes are offered.
quiz_questions	id UUID PK; quiz_id UUID FK → diagnostic_quizzes; question_text TEXT; options JSONB; correct_answer VARCHAR(255); position INTEGER. position defines display order. correct_answer is never serialized to the student client (see §7).
quiz_attempts	id UUID PK; quiz_id UUID FK → diagnostic_quizzes; student_id UUID FK → users; booking_id UUID NULL FK → bookings; score NUMERIC(5,2); submitted_at TIMESTAMPTZ. Indexes: student_id, quiz_id.

Key schema decisions

- Identifiers: UUIDs are preferred for externally visible records such as users and bookings.
- Time: Store timestamps in UTC (TIMESTAMPTZ) and display them in the configured application/user time zone.
- Booking interval: start_at and end_at are NOT NULL and CHECK (end_at > start_at). The API derives end_at from start_at + durationMinutes.
- Booking status: pending, confirmed, rejected, cancelled, completed.
- Rescheduling: use an auxiliary reschedule_requests table with id UUID PK, booking_id UUID FK, requested_start_at TIMESTAMPTZ, requested_end_at TIMESTAMPTZ, status VARCHAR(20), and created_at TIMESTAMPTZ. The original confirmed booking remains confirmed and keeps its slot until the admin approves the new time. This auxiliary table is omitted from Figure 3 for clarity.
- Calendar synchronization: Store calendar_event_id and calendar_sync_status so external failures are visible and retryable.
- File data: Keep file bytes on storage; PostgreSQL stores only path, original filename, MIME type, size and ownership.
- Indexes: Index bookings by student_id, status, start_at and topic_id; index quiz attempts and file ownership fields.
- Referential integrity: Use foreign keys and NOT NULL/UNIQUE/CHECK constraints where applicable.

Conflict prevention

The database must protect the most important scheduling invariant: two confirmed lessons cannot overlap. The API checks availability before confirmation, and PostgreSQL provides a final constraint so concurrent requests cannot bypass the rule.

POSTGRESQL — EXCLUSION CONSTRAINT

```
-- Example for a single-teacher v1 deployment
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE bookings
ADD CONSTRAINT no_overlapping_confirmed_bookings
EXCLUDE USING gist (
    tstzrange(start_at, end_at, '[') WITH &&
)
WHERE (status = 'confirmed');
```

Reasoning

Frontend availability is a convenience, not a safety control. The final conflict check belongs on the server/database because multiple clients can submit requests at nearly the same time. The `btree_gist` extension is included so the same constraint can later be scoped per teacher (adding `teacher_id WITH =`) without a redesign.

05 API Design

The REST API is the contract used by both Netlify frontends.

Method	Path	Purpose	Auth
POST	/api/auth/register	Create student account	No
POST	/api/auth/login	Authenticate and set secure auth cookie	No
POST	/api/auth/logout	Clear authentication	Yes
GET	/api/auth/me	Return current user and role	Yes
GET	/api/topics	List active physics topics/levels	No
GET	/api/availability?date=YYYY-MM-DD	List selectable lesson slots	No
POST	/api/bookings	Create pending booking	Student
GET	/api/bookings/me	List current student bookings	Student
GET	/api/bookings/:id	Read owned booking details	Owner/Admin
PATCH	/api/bookings/:id/cancel	Cancel an allowed booking	Owner/Admin
POST	/api/bookings/:id/reschedule	Request a new time	Student
POST	/api/bookings/:id/files	Upload preparation file	Owner
GET	/api/bookings/:id/files	List file metadata for a booking	Owner/Admin
GET	/api/files/:fileId	Download one authorized file	Owner/Admin
DELETE	/api/files/:fileId	Remove an uploaded file before the lesson	Owner/Admin
GET	/api/quizzes/topic/:topicId	Get active diagnostic quiz (without answers)	Student
POST	/api/quizzes/:id/attempts	Submit answers and calculate score	Student
GET	/api/progress/me	Return lesson/quiz progress summary	Student
GET	/api/admin/bookings	Filter/list all bookings	Admin
PATCH	/api/admin/bookings/:id/confirm	Confirm after conflict re-check	Admin
PATCH	/api/admin/bookings/:id/reject	Reject pending booking	Admin

Method	Path	Purpose	Auth
PATCH	/api/admin/bookings/:id/reschedule	Approve/change booked time	Admin
POST	/api/admin/availability	Create availability slots	Admin
DELETE	/api/admin/availability/:id	Remove an unused slot	Admin
POST	/api/admin/bookings/:id/lesson	Create/update lesson summary	Admin
POST	/api/internal/n8n/calendar-result	Receive authenticated integration result	Internal

Example request and response

POST /API/BOOKINGS

Content-Type: application/json

```
{
  "lessonType": "zoom",
  "durationMinutes": 90,
  "language": "ar",
  "topicId": "<uuid>",
  "startAt": "2026-08-18T14:00:00Z",
  "notes": "Exam preparation - Kirchhoff
laws"
}
```

201 CREATED

```
{
  "id": "<uuid>",
  "status": "pending",
  "lessonType": "zoom",
  "durationMinutes": 90,
  "startAt": "2026-08-18T14:00:00Z"
}
```

ONE ERROR SHAPE EVERYWHERE — 409 CONFLICT

```
{
  "error": {
    "code": "BOOKING_CONFLICT",
    "message": "The selected lesson time is no longer available.",
    "details": {}
  }
}
```

Status-code policy

200 success, 201 created, 400 invalid input, 401 unauthenticated, 403 forbidden, 404 not found, 409 conflict, 413 upload too large, 422 semantically invalid request, and 500 unexpected server error. Frontends translate the stable `code` field rather than the English message.

06 Frontend Design

Two focused React applications share one API and one data model.

Student web routes

Route	Page	Auth
/	Landing / lesson information	No
/login	Student login	No
/register	Student registration	No
/book	Lesson type, duration, language, topic, date/time and notes	Student
/dashboard	Upcoming and historical lessons	Student
/bookings/:id	Booking details, cancellation/reschedule, files	Owner
/bookings/:id/diagnostic	Diagnostic quiz for selected topic	Owner
/progress	Lesson summaries and progress history	Student
*	Not found	—

Admin web routes

Route	Page	Auth
/login	Admin login	No
/	Admin dashboard / today / pending counts	Admin
/bookings	Booking list and filters	Admin
/bookings/:id	Review request, files, quiz result, confirm/reject/reschedule	Admin
/students	Student list and lesson history	Admin
/availability	Create/remove availability slots	Admin
/lessons/:bookingId	Write lesson summary, homework and feedback	Admin
/automation	Calendar sync / reminder status and retry actions	Admin
*	Not found	—

Component structure

STUDENT APP

```
StudentApp
├─ AppRouter
├─ AuthProvider
├─ LanguageProvider
├─ BookingWizard
│   └─ LessonTypeStep
│       └─ TopicStep
│           └─ AvailabilityPicker
│               └─ BookingReview
├─ StudentDashboard
│   └─ BookingCard
│       └─ FileUploader
│           └─ LessonSummaryCard
└─ DiagnosticQuiz
```

ADMIN APP

```
AdminApp
├─ AppRouter
├─ AuthProvider
├─ AdminDashboard
├─ BookingTable
├─ BookingDetail
├─ AvailabilityManager
├─ StudentProfile
└─ LessonEditor
```

State strategy

State	Location	Reason
Authenticated user / role	Auth context	Needed across protected routes; token itself remains in HttpOnly cookie.
Preferred language / direction	Language context	Used throughout the UI to switch translations and RTL/LTR.
Bookings / availability / quiz data	Fetches from REST API per page	Server data remains authoritative and is refreshed after mutations.
Booking wizard inputs	Local component state	Temporary form state until submission.
Filters, dialogs, menu state	Local component state	Pure UI concerns that do not belong in the database.

Multilingual and RTL design

- All visible labels use translation keys rather than hard-coded strings.
- The root document direction is set to `rtl` for Arabic/Hebrew and `ltr` for English.
- Dates/times are formatted for the selected locale while the API keeps UTC timestamps.
- Layouts avoid left/right assumptions where possible; use logical start/end alignment.
- Validation and error messages are translated in the frontend from stable API error codes.

07 Security & Non-Functionals

Authentication, authorization, validation and failure behavior are part of the design, not post-build patches.

Authentication flow

1. User submits email and password to `POST /api/auth/login` over HTTPS.
2. Express looks up the user and verifies the password against a strong hash (bcrypt or argon2id).
3. Express signs a short-lived JWT and sends it in a `Secure`, `HttpOnly` cookie.
4. Each protected request passes through authentication middleware that verifies the token and loads user identity/role.
5. Authorization middleware checks role and resource ownership before a controller can read or change data.
6. Logout clears the cookie; expired tokens require login again (a refresh-token design can be added later if needed).

Netlify / API note

Because the frontends and VPS API are on different hosts, the preferred deployment is to proxy `/api/*` from each Netlify site to the VPS API. This gives the browser a same-origin API path and reduces CORS/cookie complexity. If a direct API hostname is used instead, explicit CORS credentials and cookie SameSite/Secure rules must be tested carefully.

Authorization rules

Actor	Permissions
Student	Create bookings; read/cancel/reschedule own bookings; upload/read own files; take quizzes; view own progress.
Admin	View/manage all bookings and students; manage availability; confirm/reject/reschedule; create lesson summaries; inspect automation status.
Internal n8n	May call only dedicated internal callback endpoints with a separate shared secret or signed authentication; never receives general admin privileges.
Public	Can read landing content, active topics and public availability; cannot read personal booking details.

Validation and secrets

Concern	Decision
Server validation	Validate every body/query/path parameter; frontend validation improves UX but is not trusted.
Passwords	Store bcrypt/argon2id hashes only; never log passwords or auth tokens.
Cookies	Secure + HttpOnly; suitable SameSite policy; CSRF strategy reviewed if cross-site cookies are used.
Quiz answers	The quiz endpoint serializes questions and options only; <code>correct_answer</code> never leaves the server, and scoring happens in the API on submission.
Files	Allow-list MIME types/extensions, enforce size limit, generate server-side filenames, prevent path traversal, require authorization on download.
n8n webhooks	Do not expose unauthenticated production webhooks for privileged actions; use header/JWT/shared-secret authentication and keep editor access restricted.

Concern	Decision
Database	PostgreSQL port is not public; only the application/internal network can connect.
Secrets	DATABASE_URL, JWT_SECRET, Google credentials, Telegram token and n8n secrets live in environment variables or protected service configuration.
Logging	Log request IDs, status and errors without secrets; integration failures include enough context for retry/debugging.
Rate limits	Apply rate limits to login, registration, booking creation and file upload endpoints.

Data privacy and retention

PhysEZ stores only information needed for tutoring operations. Uploaded student material is private by default. The admin can define a retention period for old files, and account deletion should remove or anonymize personal data while preserving only records that are still operationally required. The exact retention policy is an open product decision for v1 (see OQ4).

08 Testing & Deployment

How the system will be verified and how each component reaches production.

Testing strategy

Level	Coverage	Example
Unit	Pure business rules	Duration validation; cancellation window calculation; quiz scoring; status-transition rules.
Integration	Express + PostgreSQL	Register/login; create booking; confirm booking; database conflict constraint; authorization; file metadata.
Integration	Express + mocked n8n	Calendar/Telegram webhook requests and failure/retry state.
End-to-end	Browser flow	Register → book → admin confirm → student sees confirmed state; cancel/reschedule; RTL language flow.
Security checks	Protected routes / uploads	Student cannot access admin route or another student file; invalid file type/size rejected.
Deployment smoke	Production services	Netlify pages load, API health check passes, database reachable internally, n8n production webhook succeeds.

Deployment architecture

- **Student frontend:** React/Vite deployed to Netlify; its public URL is assigned during deployment.
- **Admin frontend:** React/Vite deployed to Netlify; its public URL is assigned during deployment.
- **Backend API:** Node.js/Express on the VPS behind Nginx and HTTPS.
- **Database:** PostgreSQL on the VPS/private network; port 5432 is not publicly exposed.
- **Automation:** Self-hosted n8n on the VPS, behind its own protected reverse-proxy route/hostname.

- **Files:** Dedicated VPS upload directory/volume; not served as an unrestricted public directory.
- **External services:** Google Calendar and Telegram are accessed by n8n using credentials stored on the server.

CI/CD pipeline

PIPELINES		
Frontend repositories push to main	Backend repository push / release	n8n
→ install dependencies	→ install dependencies	workflow changes
→ lint + tests	→ lint + tests	→ test workflow
→ vite build	→ deploy/restart Express	→ activate production workflow
→ Netlify deploy	→ run safe DB migrations	→ export/version workflow JSON
	→ health check	

Environment configuration

Variable	Purpose	Location
DATABASE_URL	PostgreSQL connection string	VPS backend
JWT_SECRET	JWT signing secret	VPS backend
APP_TIMEZONE	Teacher/application display time zone	VPS backend
FRONTEND_ORIGINS	Allowed direct origins if CORS is used	VPS backend
UPLOAD_DIR	Private upload directory	VPS backend
MAX_UPLOAD_MB	Upload size limit	VPS backend
N8N_WEBHOOK_BASE_URL	Internal/protected n8n base URL	VPS backend
N8N_SHARED_SECRET	Authenticate Express ↔ n8n calls	VPS backend + n8n
GOOGLE_CALENDAR_ID	Target teaching calendar	n8n
Google OAuth credentials	Calendar API authorization	n8n credential store
TELEGRAM_BOT_TOKEN	Bot token for admin alerts	n8n credential store
TELEGRAM_CHAT_ID	Destination admin chat	n8n credential/config
VITE_API_BASE	Usually <code>/api</code> when using the Netlify proxy	Netlify build env

Backups and recovery

- Daily PostgreSQL dump or equivalent scheduled database backup, retained for a defined period.
- Regular backup/snapshot of the upload directory.
- n8n workflow exports and credentials recovery plan; credentials themselves remain encrypted/protected.
- At least one restore test before final submission so backups are known to be usable.

Timeline, Risks & Appendix

A realistic delivery plan, known risks, future scope and references.

Proposed milestones

Milestone	Includes	Target
M1 — Foundation	Repositories, React shells, Express API, PostgreSQL schema, local environments	Week 1
M2 — Auth + booking	Student registration/login, topics, availability, create/list bookings	Week 2
M3 — Admin + automation	Admin role/dashboard, confirm/reject/cancel, n8n Telegram and Calendar integration	Week 3
M4 — Student tools	Student dashboard, file uploads, rescheduling and authorization tests	Week 4
M5 — Learning layer	Diagnostic quiz, lesson summary, progress view, Arabic/Hebrew/English RTL	Week 5
M6 — Quality + deploy	Conflict tests, security review, backups, Netlify/VPS deployment, documentation and demo	Week 6

Risks and mitigations

Risk	Impact	Mitigation
Scope creep	Many tutoring features could delay a working core.	Keep v1 non-goals fixed; finish booking/auth/calendar before optional learning features.
Double-booking	Concurrent requests can make the same slot appear free.	Server re-check plus PostgreSQL overlap constraint; never trust frontend availability alone.
Calendar/API failure	External service may fail after a local state change.	Persist sync status, keep slot blocked, show retry state, let n8n retry safely.
n8n exposure	A public privileged webhook could be abused.	Authenticated webhooks, restricted editor, reverse proxy, separate internal callback endpoints.
Cookie/CORS issues	Netlify and VPS are different hosts.	Prefer the Netlify <code>/api</code> rewrite proxy; otherwise test credentialed CORS and cookie policy early.
File abuse	Large or unexpected uploads can consume disk or expose content.	Allow-list types, size limits, unique filenames, private storage, disk monitoring and backups.
VPS single point of failure	API, database, files and n8n share one machine.	Accept for course scope; maintain backups and keep future migration to managed DB/object storage possible.
Integration credentials	Google/Telegram secrets could leak through source control.	Use environment/credential stores only; rotate secrets if exposed; never include them in client code.

Future features (v2+)

- Recurring weekly lessons and lesson packages.

- Waiting list with automatic notification when a time becomes free.
- Online payments and invoices.
- Email/SMS/WhatsApp notification channels in addition to Telegram.
- Advanced admin statistics: utilization, topics, cancellation rate and revenue.
- S3-compatible object storage and CDN for files.
- Multiple teachers with independent calendars and availability.
- Optional Zoom/Google Meet link generation rather than manually stored meeting URLs.
- Parent/guardian accounts for younger students.
- More detailed learning analytics and homework submission/feedback workflows.

Open questions

ID	Question
0Q1	Will the final API use a custom domain/subdomain, or only a VPS-hosted HTTPS hostname?
0Q2	Should the student be allowed to cancel at any time, or will there be a configurable cancellation window?
0Q3	Will online bookings use one reusable Zoom link, a manually entered link per booking, or future Zoom API integration?
0Q4	What file-retention period should apply to old student uploads?
0Q5	Should diagnostic quizzes be required for selected topics or always optional?

References and implementation sources

Ref	Source	URL / note
R1	NewTech Bootcamp — Full-Stack Design Doc Template	Course material supplied with the project.
R2	React — Build a React app from Scratch	react.dev/learn/build-a-react-app-from-scratch
R3	Express — Production Best Practices: Security	expressjs.com/en/advanced/best-practice-security/
R4	PostgreSQL — Constraints	postgresql.org/docs/current/ddl-constraints.html
R5	Netlify — Vite on Netlify	docs.netlify.com/build/frameworks/framework-setup-guides/vite/
R6	Netlify — Rewrites and proxies	docs.netlify.com/manage/routing/redirects/rewrites-proxies/
R7	n8n — Host n8n / self-hosting	docs.n8n.io/deploy/host-n8n
R8	n8n — Configure webhook URLs with reverse proxy	docs.n8n.io/deploy/host-n8n/configure-n8n/basic-configuration/
R9	Google Calendar API — Create events	developers.google.com/workspace/calendar/api/guides/create-events
R10	Google Calendar API — Authorization scopes	developers.google.com/workspace/calendar/api/auth

References and implementation sources (continued)

Ref	Source	URL / note
R11	Telegram — Bot API	core.telegram.org/bots/api
R12	NGINX — HTTP proxy module	nginx.org/en/docs/http/ngx_http_proxy_module.html
R13	Certbot — Nginx HTTPS instructions	certbot.eff.org/instructions